Noname manuscript No.
(will be inserted by the editor)

A Model-Driven Approach to Simulate Queuing Network Models

Javier Troya, Antonio Vallecillo

GISUM/Atenea Research Group. Universidad de Málaga (Spain)
e-mail: {javiertc,av}@lcc.uma.es

The date of receipt and acceptance will be inserted by the editor

Abstract Queuing Network Models (QNMs) provide powerful notations and tools for modeling and analyzing the performance of many different kinds of systems. However, QNMs are quite restrictive regarding the probability distributions that they can handle: basically exponential and uniform distributions. The common way to address this limitation is by using simulation, although this solution is not free from problems either. Simulation is normally expensive and developed in an ad-hoc and low level manner for every system that needs to be analyzed. In this paper we propose a new Model-Driven approach to simulate QNMs where arrival and service times can follow arbitrary probability distributions. We define one high-level and generic behavioral model for QNMs that can be simulated using standard model transformation techniques. Then, QNMs specified using the Performance Model Interchange Format (PMIF) can be automatically translated into our representation and simulated for obtaining their performance indicators.

Key words Queuing Network Models – PMIF – Simulation

1 Introduction

Non-functional requirements are often difficult to capture, measure and predict. They concern the quality of resources of a system, how good a design of a system is, etc. In many occasions, they are critical for success. Among non-functional requirements, performance is one of the most influential factors to be considered. Performance problems in systems development may be so severe that they can require considerable changes in design. In the worst cases, performance issues can even impact the requirements level. Furthermore, distributed software normally includes stochastic behavior, so that the obtained results are different in different executions.

Historically, the approach to carry out performance analysis was to export performance modeling and measurements from the hardware domain to software systems. It assumed a new dimension when the focus was directed towards software applications. As a first attempt, software was developed using a "fix-it-later" approach, where performance issues were introduced later in the development process. However, with the increase of software complexity, it came out that software performance could not be managed locally at the code level since performance problems often result from early design choices. With the objective of identifying performance problems in earlier stages of the software development, several approaches have been proposed. In this way, many specification formalisms provide corresponding extensions including stochastic transition systems (or Markov Chains [1]), like stochastic Petri Nets [2,3], Process Algebras [4,5] or Queuing Network Models [6].

Among all these formalisms, this paper focuses on Queuing Network Models (QNMs). Queuing theory is the key analytical modeling technique used for computer systems performance analysts. In computer systems, many jobs share system resources such as CPUs, disks, memory, and other devices. Generally, one job (or some of them) can use a resource at any time, so the remaining jobs need to wait to use that resource in queues. Queuing theory helps in determining the time that the jobs spend in various queues in the system. These times can then be combined analytically to predict the response time, which is basically the total time that the job spends inside the system, and other non-functional properties. Queuing models have become very popular among computer system performance analysts. In many cases, jobs flow among resources in a non-deterministic manner, so stochastic methods are needed to analyze such systems.

The literature on queuing theory is vast, and many papers are published every year. At the same time, some tools to analyze queuing network models exist, for example SHARPE [7], PEPSY/QNS [8], PDQ Analyzer [9], MVA Queuing Formalism Parser [10], MQNA [11], QNAP [12] or MOSEL [13]. Most of them do not allow to graphically draw the models, so users have to insert them in the tools' models representation. These tools use analytical models as opposite to simulation. The best advantage of analytical models with regards to simulation is the time consumed to obtain the results. Nevertheless, analytical modeling generally requires many simpli-

fications and assumptions. Simulations can incorporate more details and require less assumptions than analytical modeling, and more often are closer to reality. In this way, an analytical model may be accurate if the arrival time of jobs follows an Exponential distribution. However, if the arrival rate changes for example to a Weibull distribution, the model is no longer valid. A simulation, instead, may admit many probabilistic distributions.

Another issue concerning queuing network analysis and the associated tools has to do with the input format these tools require. In fact, each tool has a different model representation. To address the problem of exchanging models among tools, a performance model interchange format (PMIF) [14–17] has been proposed. PMIF is a common representation for system performance model data that can be used to move models among modeling tools that use a QNM paradigm. The idea behind this approach is that a user of several tools that support the format can create a model in one tool and later move the model to another tool without the need to translate it to the other tool's representation model.

In this paper we propose a generic behavioral model for QNMs specified in terms of behavioral rules. This behavioral model can then be simulated so that performance metrics for QNMs are obtained. This approach accepts many probabilistic distributions for arrival and service times (Exponential, Uniform, Geometric, Weibull, Normal, Lognormal, Gamma, etc.). We also present a tool where QNMs can be graphically depicted and later translated to our behavioral representation, so that QNMs depicted by the user can then be simulated. QNMs created with our tool conform to an extension of the PMIF metamodel. The behavioral model that we present here also serves to provide a behavioral semantics for QNMs, by expressing how they work using behavioral rules.

The rest of the paper is organized as follows. Section 2 presents the state of the art regarding QNMs and PMIF. Section 3 introduces essential concepts on queuing theory and presents our new approach for PMIF. Then, Section 4 presents the MDE concepts needed to understand our MDE approach for giving behavior to QNMs. Section 5 describes the generic behavioral model for QNMs. The next section deals with the different mechanisms for describing QNMs to later obtain their behavioral models. Section 7 explains how we deal with QNMs behavioral simulations and finally, in Section 8, we expose the conclusions and present some future work.

2 State of the Art

In this section we expose the state of the art of the two most important issues of this paper: QNM analysis and PMIF. In fact, a point that we address in this work is to combine both so that PMIF models can be simulated with our tool.

2.1 Queuing Network Models for Performance

Queuing network models have been extensively used to study the performance of existing and planned systems in application areas such as communications networks, computer systems, manufacturing processes, transportation systems or customer service facilities [18]. There are several commercial packages to queuing network modeling, like QNAP [12], RESQME [19], BEST/1, CSIM. There are also many academic tools including TANGRAM-II [20,21], SHARPE [7] or JINQS [22].

Most of these packages and tools were developed some years ago. As a consequence, none of them is prepared to receive a PMIF model as input. In fact, each of them specifies a queuing network model in a different way and with a different language, and few of them provide a graphical interface for the definition of QNMs. Furthermore, many of them provide analysis for a specific type of queuing networks, but cannot do anything with the rest. For example, RESQME provides a graphical environment for constructing, solving and analyzing the results of extended queuing network models of resource contention systems. Despite its graphical environment, which is very helpful, a model depicted in this tool also needs to contain textual information. This tool presents the performance results in a very interesting manner. Thus, when the solution is complete, the user can point to any desired node in the model diagram to specify performance measures which are then displayed in graphical and tabular forms.

TANGRAM-II was developed for research and educational purposes. The first version of TANGRAM [23] was developed in Prolog and a graphic interface called TGIF [24] was also implemented and became later a full fledge independent general purpose sophisticated drawing tool. Queuing network models are specified by means of objects and events. Not every model can be solved analytically in this tool. In fact, only if the model is Markovian or belongs to a class of non-Markovian models; otherwise, it is solved via simulation. The simulator is implemented in C++.

Another direction explored recently is the idea of considering fast queuing network approximations at the stochastic process level by means of linear programming. An advantage of these approaches over simulation is that linear programming can accurately describe hundreds of thousands or even millions of state probabilities without severe scalability issues. In the lp-rBm technique, a queuing network can be described as a multidimensional reflected Brownian motion (rBm), following several related approximations proposed from the early 80's. The rBm approach is very powerful to represent non-exponential distributions, as it relies on a second-order theory which explicitly considered variances and covariances between service times and queue lengths. Its major shortcoming is that it does not apply effectively to closed models. In our work, we cover open and closed models.

Some tools based on simulation need to know when the steady state of the simulation starts, i.e., the warm-up period must be specified by the user and will typically be based on observations of pilots of the model. An example of such tools is JINQS, where there is no built-in mechanism for detecting when a simulation is close to a steady state. In this tool, when the approximate warm-up period is known, the simu-

late method can be optionally parameterized by this warm-up period. In our tool we have applied mechanisms to identify when the results have reached a steady state and the simulation can be stopped.

Most of these tools and packages do not support arbitrary probabilistic distributions for arrival and service times, but only Exponential and Uniform distributions. Otherwise solving the analytical models is not possible. This is why some packages also offer solutions based on simulation, e.g., TANGRAM-II and JINQS. They allow solving QNMs that use different probabilistic distributions (Exponential, Deterministic, Uniform, Erlang, Log-normal, Pareto, etc.) using simulation packages built-in in their tools. Our tool also accepts the most commonly used probabilistic distributions (Uniform, Exponential, Normal, Gamma, Weibull, Erlang, F, ChiSquare, Geometric, Log-normal, Pareto, Pascal and Poisson) for arrival and service times. Furthermore, our approach to simulate QNMs behavior is different from those existing today. We apply an Model-Driven Engineering (MDE) approach to design and simulate the models, in a flexible manner. We can also deal with both open and closed queuing models.

2.2 Evolution of PMIF

PMIF was conceived as a common representation for system performance model data than can be used to move models among modeling tools [17]. Its creators were interested in tool inter-operability for Software Performance Engineering (SPE). SPE is a method for constructing software systems that meet performance goals [25]. Its structure represents the software processing steps and other information for workloads that execute on the system performance level. PMIF, however, was born for system performance models that represent computer platforms and network interconnections with a network of queues and servers. Its representation technique had to be appropriate to express the interchange format and it needed to be capable of expressing a wide range of system execution models: those containing a small number of servers to very large number of them, from one to many workloads, both open and closed models, that may be solved using either analytic or simulation solution techniques... It also had to be useful with existing tools, include modeling features that tools provide, support the modeling paradigms prevalent in tools, and use terminology common in tools and modeling research. In this way, the first version of PMIF (1998), as explained in [17], addressed a specific type of performance model: Queuing Network Models that may be solved using exact analytical solution algorithms. This version omitted graphics, and the resulting metamodel for QNMs can be seen in Figure 1.

In this version, it was proposed to use the operational analysis term visits rather than the stochastic modeling probability among servers. It was designed using an EIA/CDIF (Electronic Industries Association/CASE Data Interchange Format) paradigm [26] that calls for refining the information requirements for a QNM with a metamodel, i.e., a model of the information that goes into constructing a QNM.

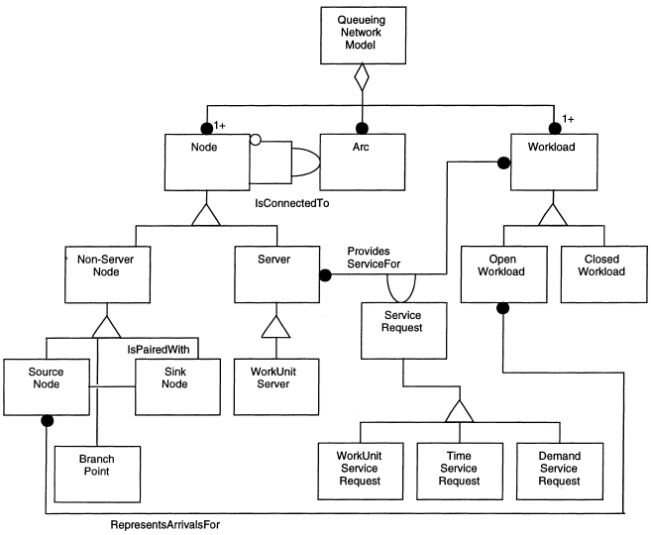


Fig. 1 PMIF 1.0 Metamodel.

A new version of the PMIF metamodel and its XML schema specification (called PMIF 2.0) was then presented in [15,27, 16]. An XML-based approach was used to tackle the complexity and amount of effort required to create the PMIF interface [28]. It uses the previous PMIF (PMIF 1.0) metamodel as a starting point because it is a good description of the information requirements for performance model interchange, but uses XML to implement the transfer format. As previously mentioned, the PMIF 1.0 metamodel uses number of visits instead of routing probabilities, assuming that from the number of visits, and with the knowledge of the queuing network topology, routing probabilities can be calculated. This assumption is true for many of the queuing networks that model computer systems. However, it is not true for the general case. This is why the routing probability was added as a transit element which specifies where a job has to transit and with what probability.

The latest version of PMIF (called PMIF 2) is described in [14]. They define the semantic properties for a pmif.xml interchange, the prescribed validation order, errors and warnings, and a tool and Web service implementation. Import and export prototypes for two different types of tools prove the concept. The metamodel for PMIF 2 is shown in Fig. 2.

In this paper, we take a step forward in the development of a general metamodel for QNMs. The metamodels shown before are not rigorous enough, in an MDE context, in the sense that they are not developed conforming to any metametamodel. In fact, their aim is to create XML files that represent pmif models and whose structure conforms to the metamodel. In our case, we apply an MDE methodology for the definition of the PMIF metamodel. In this way, the metametamodel of our metamodel is ECORE, and so ECORE models representing different queuing network models can be defined. Furthermore, many probabilistic distributions for arrival and service times can be specify in the models. Our approach is further explained in the next section.

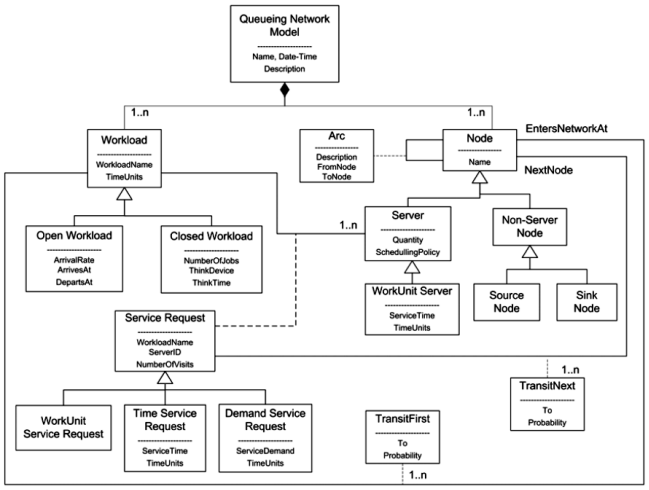


Fig. 2 PMIF 2 Metamodel (from [14])

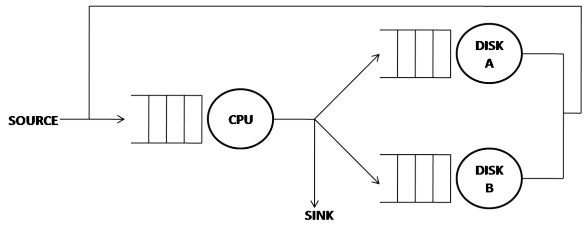


Fig. 3 Open Queuing Network

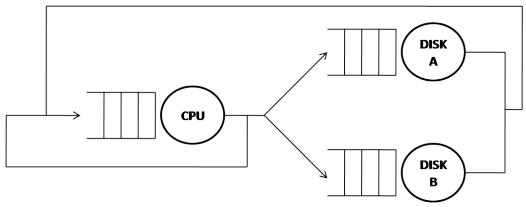


Fig. 4 Closed Queuing Network

3 Queuing theory and PMIF

In computer systems, many jobs share the system resources such as CPU, disks, and other devices. Since generally only one job (or some of them) can use the resource at any time, all other jobs wanting to use that resource wait in queues. Queuing theory helps in determining the time that jobs spend in various queues in the system. These times can then be combined to predict the response time, which is basically the total time that the job spends inside the system, and many other non-functional features. Systems where jobs may receive service at one or more queues before exiting from the system are modeled with queuing networks. In general, a model in which jobs departing from one queue arrive to another queue (or possibly the same queue) is called a queuing network.

There are two types of queuing networks: open and closed. The former ones have external arrivals and departures. The jobs enter the system at a source and depart at a sink (Fig. 3). The number of jobs in the system varies with time. The latter ones have no external arrivals or departures: the jobs in the system keep circulating from one queue to the next. The total number of jobs in the system is constant. It is possible to view a closed system as a system where the sink is connected back to the source (Fig. 4). The jobs exiting the system immediately reenter the system.

The ECORE metamodel we propose for defining QNMs, named xPMIF (for *extended*-PMIF), is shown in Figure 5. It can be seen as the MDE version of the PMIF 2 metamodel presented in [14] (Fig. 2), with some slight extensions.

A QueuingNetworkModel is composed of one or more Workloads, zero or more Arcs, one or more Nodes and one or more ServiceRequests. The Arc class connects Nodes among them. In a queuing network, jobs flow from node to node. There are two types of nodes, Servers and NonServerNodes. The former ones provide some processing service, while the latter show the topology of the network and represent the origin (SourceNode) and exit point (SinkNode) of OpenWorkloads. The Server class has a specialization class, named WorkUnitServer, that represents resources with a fixed processing service for each Workload that makes a request for service.

A Server provides service for different Workloads, where a Workload represents a collection of jobs that make similar ServiceRequests from Servers. Depending on the type of queuing network (open or closed), there are two types of Workloads:

- OpenWorkload. It represents a set of jobs which arrive from the outside world, receive service, and exit. The number of jobs belonging to an OpenWorkload at any time is variable. A job represented by an OpenWorkload arrives at a SourceNode and departs at a SinkNode.
- ClosedWorkload. It represents a fixed population of jobs that circulates among the Servers. A ClosedWorkload has a Server which acts as ThinkDevice and which is characterized by a think time.

A ServiceRequest associates Workloads with Servers. According to the relation from ServiceRequest to Workload and Server (one to one in both cases), a ServiceRequest associates one (and only one) Server with one (and only one) Workload. In this way, when a job which is represented by a workload arrives to a server, the service request associated to the workload and the server specifies how the job will be treated in that server. There are three types of ServiceRequests (for all of them, the numberOfVisits is an optional attribute):

- WorkUnitServiceRequest. They are ServiceRequests associated to WorkUnitServers, so nothing about the service time has to be specified, since it is already in the WorkUnitServer.
- TimeServiceRequest. It specifies the service time that the jobs representing the Workload associated to the ServiceRequest will have in the associated Server.
- DemandServiceRequest. Same, but service time is specified in terms of service demand (service time multiplied by number of visits).

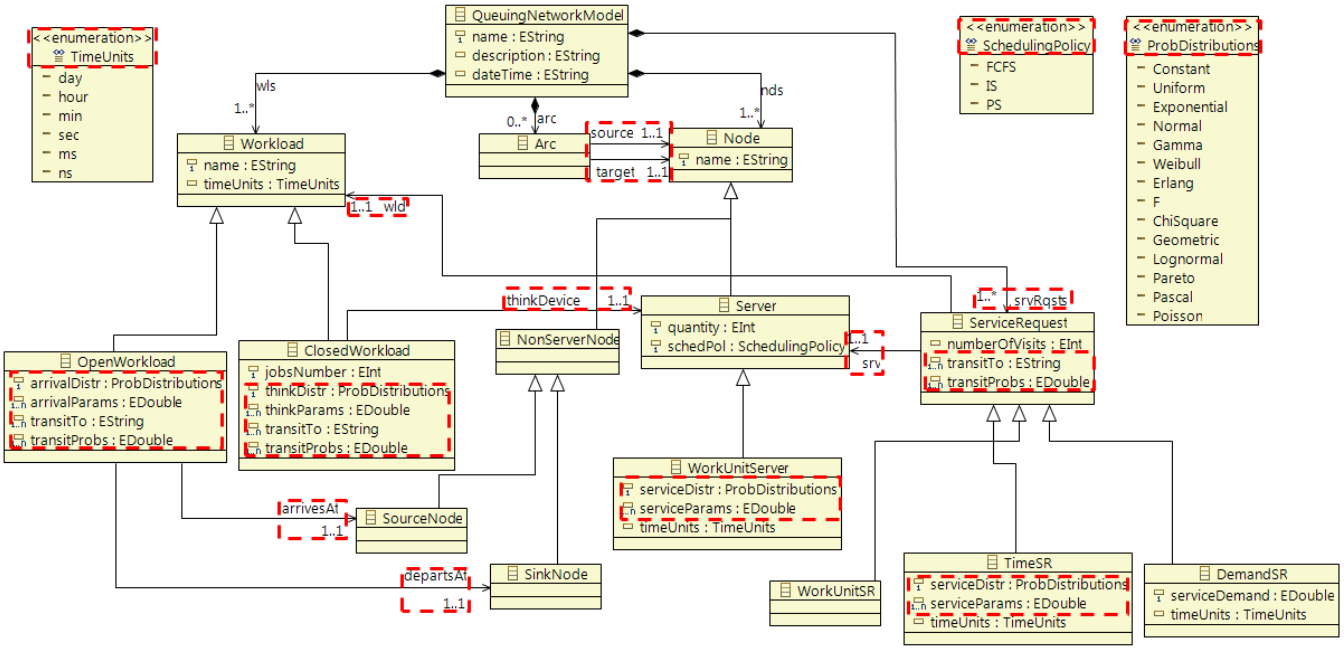


Fig. 5 xPMIF metamodel conforming to ECORE

So far, the features described of our metamodel are similar to those of PMIF 2 (Fig. 2). The differences are the following (they are marked with dotted boxes in Fig. 5). In PMIF 2, probabilities are specified as classes in the metamodel, and then those elements that need them (ServiceRequests and Workloads) have a reference to those classes. In a QNM where there are many forks in many servers, this approach results in a model with many elements. To reduce this number of elements, probabilities are specified as attributes in our metamodel. There are two classes that specify transitions: Workload and ServiceRequest. Probabilities in the former specify where a job transits after entering the SourceNode. In the latter, probabilities specify where a job transits after being processed by the Server associated with the ServiceRequest. In the PMIF 2 metamodel, an element of type Transit was needed for each path in a fork. In our case, no matter how many paths a Workload may follow, we only need two attributes: transitTo and transitProbs. The former contains a sequence (the order matters) with the names of the Nodes where the Workload can transit. The latter contains a sequence with the probabilities of transitions to the Servers contained in the transitTo attribute. Please note that the order in both attributes has to match.

As an example, suppose that in the queueing network model in Fig. 3, a ServiceRequest associated to the CPU could contain the following two attributes for specifying the transitions: transitTo: Sequence(DISKA, DISKB, SINK) and transitProbs: Sequence(0.4375, 0.5, 0.0625) (example taken from [29, page 572]). It means that the probability to transit to the DISKA is 0.4375, it is 0.5 to transit to DISKB, and it leaves the system with a probability of 0.0625. The addition of the values in the transitProbs sequence must be 1. In the transitions where there is only one path, the sequences of both attributes only contain one element.

Another difference is how service and arrival times are specified. In PMIF 2, they were specified with the attributes ArrivalRate and ServiceTime respectively. It is assumed that these values represent parameters of Poisson and Exponential distributions, respectively. In our case, we want PMIF to accept different probabilistic distributions for service and arrival times. In this way, we have a type (ProbDistributions) which is an enumeration with literals Constant, Uniform, Exponential, Normal, Gamma, Weibull, Erlang, F, ChiSquare, Geometric, Lognormal, Pareto, Pascal and Poisson. Our idea to specify arrival and service times is to use two attributes: arrivalDistr or serviceDistr, and arrivalParams or serviceParams. The former ones are of type ProbDistributions and specify the distribution that will model the arrival/service time. The latter ones contain a sequence (the order matters) with the parameters of such distributions. To apply an order to the parameters in the sequence, these will be ordered according to the alphabetical order of the name of the parameters. In this way, the parameters of the distributions are the following:

- Constant(double time)
- Uniform(double lowerBound ; double upperBound)
- Exponential(double mean)
- Normal(double mean ; double variance)
- Gamma(double scale ; double shape)
- Weibull(double scale ; double shape)
- Erlang(double scale ; double shape)
- F(double denominatorDegreesOfFreedom ; double numeratorDegreesOfFreedom)
- Chi-square(double degreesOfFreedom)
- Geometric(double probOfSuccess)
- Lognormal(double logScale ; double shape)

- Pareto(double scale ; double shape)
- Pascal(double numOfSuccesses ; double probOfSuccess)
- Poisson(double mean)

If the distribution is Constant, it means that the arrival/service time is constant.

The last significant difference with PMIF 2 is the use of references instead of attributes referring to names in some classes. Thus, for example, class Open Workload has attributes ArrivesAt and DepartsAt in the PMIF 2 metamodel which are of type String. They specify the name of the Source Node and Sink Node, respectively, that they are referring to. The same happens with class Service Request and attributes WorkloadName and ServerID. In our metamodel these attributes are references. This has the advantage that a reference cannot be incorrectly specified since a reference to an object that does not exist or to an object belonging to a different class from the one specified in the metamodel cannot be created, but a name, instead, can be improperly written.

4 MDE Essentials

MDE [30] is becoming a widely accepted approach for developing complex distributed applications. MDE advocates the use of models as the key artifacts in all phases of development, from system specification and analysis, to design and implementation. Each model usually addresses one concern, independently from the rest of the issues involved in the construction of the system. A very important MDE concept is Domain Specific Languages (DSLs).

Domain Specific Languages play a cornerstone role in Model-Driven Engineering for representing models and metamodels. The benefits of using DSLs is that they provide an intuitive notation, closer to the language of the domain expert, and at the right level of abstraction.

In this work we make use of DSL models for two purposes. The first one is to provide the user with a friendly user interface to specify and depict queuing network models (Section 6). We have used the Graphical Modeling Framework (GMF [31]) for this purpose. The second one is for implementing and analyzing queuing network models by means of its behavioral modeling (Section 5).

4.1 The Graphical Modeling Framework.

GMF provides a generative component and runtime infrastructure for developing graphical model editors for DSLs. It automatically generates an Eclipse plugin with a DSL diagram editor from (1) the DSL metamodel (abstract syntax); (2) a graphical definition (concrete syntax); (3) a tooling definition (i.e., the buttons that enable the creation of the model elements); and (4) a mapping model relating the three previous artifacts.

The abstract syntax of a DSL is normally specified by a metamodel, which describes the concepts of the language, the relationships among them, and the structuring rules that constrain the model elements and their combinations in order to represent the domain rules. In our case, it is the metamodel shown in Fig. 5. The concrete syntax of a DSL provides a realization of the abstract syntax of a metamodel as a mapping between the metamodel concepts and their textual or graphical representation. For visual representations, as in our case, it is necessary to establish links between these concepts and the visual symbols that represent them. In this way, we have tried to select those visual symbols so that they are as intuitive as possible for the user. Some of them, like the one used for a server, have been widely used ever since queueing models started to be studied. Others, like service requests, are new concepts that come from the approach to standardize queueing network models with PMIF.

4.2 Modeling the Behavior of DSL models.

One way of specifying the dynamic behavior of a DSL is by describing the evolution of the modeled artifacts along some time model. In MDE, this can be done using model transformations supporting in-place update [32]. The behavior of the DSL is then specified in terms of the permitted actions, which are in turn modeled by the transformation rules.

The environment we have used for specifying the behavior of queueing network models, named *e-Motions* [33,34], is a Domain Specific Language (DSL) and graphical framework developed for Eclipse that supports the specification, simulation, and formal analysis of real-time systems.

e-Motions extends in-place transformation rules with a quantitative model of time and with mechanisms that allow designers to specify action-based properties, thus facilitating the design of real-time complex systems. While there are several approaches that propose in-place model transformation rules to deal with the behavior of DSLs from textual to graphical [35], *e-Motions* provides a very intuitive and natural way to specify behavioral semantics, close to the language of the domain expert and the right level of abstraction [36]. These transformations are composed of a set of rules, each of which represents a possible *action* of the system. These rules are of the form $l : [\text{NAC}]^* \times \text{LHS} \rightarrow \text{RHS}$, where l is the rule's label (its name); and LHS (left-hand side), RHS (right-hand side), and NAC (negative application conditions) are model patterns that represent certain (sub-)states of the system. The LHS and NAC patterns express the precondition for the rule to be applied, whereas the RHS one represents its postcondition, i.e., the effect of the corresponding action. Thus, a rule can be applied, i.e., triggered, if an occurrence (or match) of the LHS is found in the model and none of its NAC patterns occurs. Generally, if several matches are found, one of them is non-deterministically selected and applied, producing a new model where the match is substituted by the appropriate instantiation of its RHS pattern (the rule's *realization*).

The model transformation proceeds by applying the rules in a non-deterministic order, until none is applicable — although this behavior can be usually modified by some execution control mechanism [35].

e-Motions allows attributes to be added to rules to represent features like duration or periodicity. There are two types of rules to specify time-dependent behavior, namely *atomic* and *ongoing* rules. The former ones represent atomic actions, with a specific duration. They can be canceled, but cannot be interrupted. These rules can be periodic, i.e., they admit a parameter that specifies an amount of time after which the action is periodically triggered (if the rule's precondition holds, of course). *Ongoing* rules represent interruptible continuous actions which progress along time while the rule preconditions (LHS and not NACs) hold. Both atomic and ongoing rules can be scheduled, or be given an execution interval. OCL [37] expressions can be used to specify values for attributes, variables, conditions, etc.

It is also possible to explicitly represent *action executions*, which describe actions currently executing. They specify the type of the action (i.e., the name of the atomic rule), the identifier of the action execution, its starting and ending time, and the set of objects involved in the action. This provides a very useful mechanism when we want to check whether an object is participating in an action or not, or if an action has already been executed, for instance.

Finally, a special kind of object, named Clock, represents the current global time elapse. Designers are allowed to use it in their timed rules (using its attribute time) to look up the amount of time that the system has been working. Further clocks can be specified by users, according to the requirements of their systems (to model, for instance, systems with distributed clocks).

5 A Generic Behavioral Model for QNMs

In this section we explain how the behavior of queuing network models can be specified with *e-Motions*. Then, in the next section, we present our tool called xQNM (*executable*-QNM) to draw QNMs which are the input models to which behavior is given. In Fig. 6 we can see the whole approach for the tool and the QNM behavior implementation. This section corresponds to the ovals numbered 4 and 5 in the figure.

5.1 QNMs structural model

When we define a behavioral model in *e-Motions*, we first need to specify the structural model of the system (oval 4 in Fig. 6). The structural model is a metamodel, conforming to ECORE, to which the models depicted in the behavioral rules must conform.

In our approach, QNMs are modeled using the xPMIF metamodel (Fig. 5). Of course, being xPMIF an extension of PMIF 2 (Fig. 2), PMIF 2 models can be used too — in xPMIF we tried to respect the structure of PMIF 2 precisely for backwards compatibility reasons.

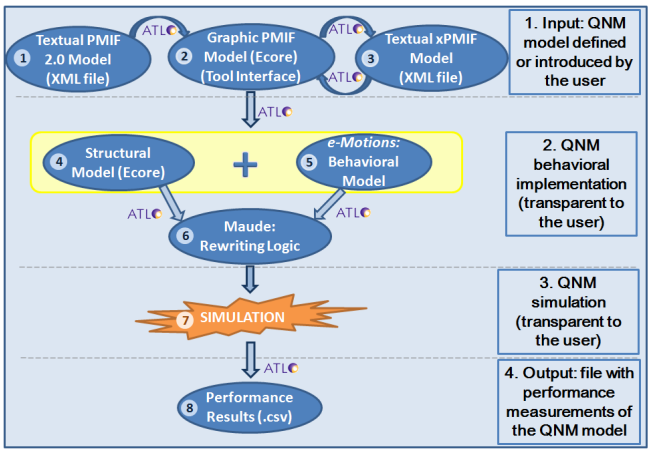


Fig. 6 xQNM Architecture

However, we realized that we need to optimize this representation to make it more compact and efficient for internal handling in *e-Motions*. Such new representation is internal to our tool and transparent to users, who still use xPMIF models to describe their QNM models. This metamodel, that we shall refer to as eMotions-xPMIF, is shown in Figure 7. Although all main classes are the same in both metamodels (QueuingNetworkModel, Workload, Node and ServiceRequest — now TServiceRequest), eMotions-xPMIF contains some changes to optimize the implementation of QNMs behavior in *e-Motions*:

- In xPMIF, transitions are specified for objects Workload (both OpenWorkload and ClosedWorkload) and ServiceRequest. In eMotions-xPMIF, there are no transitions for ClosedWorkload objects because they are not needed. As for OpenWorkload objects, we have made an specialization: OWL1T for OpenWorkloads that always enter the same server and OWLnT for OpenWorkloads when there is more than one server to which transition is possible when the jobs enter the system. The same thing happens with ServiceRequests, where the same specialization has been done. This specialization has been done to improve the efficiency of the rules shown in Section 5.3.
- In eMotions-xPMIF, we have only one type of ServiceRequest instead of three. It is TServiceRequest (for "time service request"). It is due to some reasons. Firstly, if we have only one type of objects for service requests, then we need fewer behavioral rules to model the system and, consequently, it is more efficient. Secondly, WorkUnitServiceRequest objects can be directly converted to TimeServiceRequest objects as long as WorkUnitServer objects are converted to Server objects. Finally, we can also convert DemandServiceRequest objects to TimeServiceRequest objects because the service time of the TimeServiceRequest can be obtained dividing the service demand of the DemandServiceRequest by the number of visits.
  Now, being all service requests of type TServiceRequest, we can see that this class has many attributes. The attributes serviceDistr and serviceParams are those used for

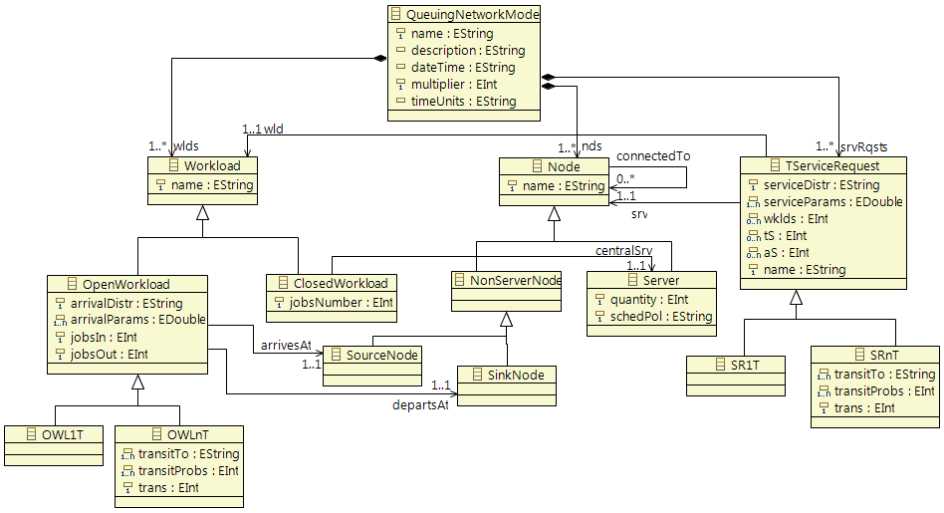


Fig. 7 Internal QNMs Structural Model for *e-Motions*

service times of jobs, as explained in Section 3. Attributes wklds, tS and aS are using for jobs flow, as explained in Section 5.3.

- There are no WorkUnitServer objects anymore, they are all of type Server now. This is directly correlated to the previous point: if there are no WorkUnitServiceRequests, then we do not need WorkUnitServers (and viceversa).
- In ClosedWorkloads, we do not specify parameters for think time. The reason is that the thinkDevice of a ClosedWorkload is nothing but a Server, just as the rest of servers. As a consequence, we have considered that the think time (i.e, its probability distribution and parameters) is specified by a TServiceRequest, just like for the other Servers. However, we do have a centralSrv reference to a Server across which all jobs have to flow. Its use is shown in Section 5.3.
- Attributes transitProbs are not of type double anymore but of type integer. They contain the probabilities in a scale from 1 to 10000. They represent the same probabilities as in the xPMIF metamodel, but multiplied by 10000. Furthermore, in this new attribute the probability in position n in the sequence contains the value in the position n in the sequence in xPMIF plus the addition of all the previous values (all multiplied by 10000). This representation improves the efficiency of transitions in the simulation.

5.2 Adding Observers to the structural model

Starting from the metamodel for *e-Motions* described before, we can design behavioral rules that specify the dynamics of QNMs in terms of jobs flow. Now, we introduce our proposal to also monitor performance indicators for QNMs in our behavioral rules. To calculate the value of these simulation parameters we propose the use of *observers* [38]. An observer is an object whose purpose is to monitor the state of the system: the state of the objects, of the actions, or both. Observers, as any other objects, have a state and a well-defined behavior. The attributes of the observers capture their state, and are used to store the variables that we want to monitor.

To introduce observers in the behavioral rules, we need to specify a metamodel for them. It is shown in Fig. 8. The idea is to combine both metamodels (Figs. 7 and Fig. 8) so that observers can be used in our behavioral rules. In fact, since *e-Motions* allows users to merge several metamodels in the definition of a DSL behavior, we can define the observers metamodel in a non-intrusive way, i.e., we do not need to modify the system metamodel to add observers in their rule. Furthermore, this approach also enables the reuse of observers across different DSLs.

In the observers metamodel we can see that there are three types of observers, each of which has been designed to monitor the performance metrics of a different type of object. In this way, we have WorkloadOb for monitoring Workloads, ServerOb to monitor Servers, and ServiceRequestOb to monitor TServiceRequests. All of them have a reference to the class EOject, which specifies that they can refer to any object.

The aim of WorkloadOb observers is to monitor performance properties of workloads. The idea is to associate an observer of this type to each workload. Their attributes are (please note that when we refer to jobs, due to the possibility of having more than one type of workload, we mean jobs belonging to the monitored workload):

- thoughput. It measures the throughput of the workload. In open workloads, it is calculated as the number of jobs that leave the system per unit of time. In closed workloads, it refers to the number of jobs that complete a cycle within the network per unit of time.
- respTimeAcc. It is used to calculate the response time of the workload. It stores the addition of response times of all the jobs.
- respTimeAv. It stores the response time. It is calculated as the average of the response times of all the jobs.

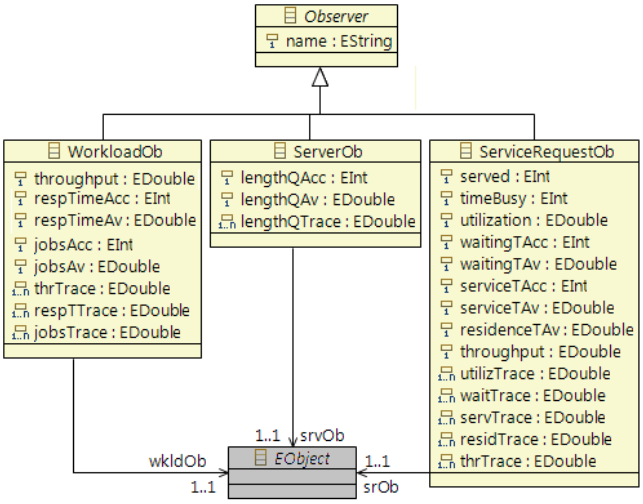


Fig. 8 *Observers* metamodel

- jobsAcc. This attribute is used to compute the mean number of jobs in the network.
- jobsAv. It calculates the mean number of jobs in the network.
- thrTrace, respTTrace and jobsTrace. They store a trace with the values for throughput, response time and jobs average, respectively, at different times of the simulation. They are used for determining when to stop the simulation.

ServerOb observers deal with the monitoring of servers. They basically monitor the queue length in the attribute lengthQAv, considering the jobs belonging to any kind of workloads that can request service in the server, and keep its trace in the attribute lengthQTrace. Attribute lengthQAcc is used for the calculation of lengthQAv. As explained in [29], the queue length of a server considers the jobs in the queue and the jobs being served.

Each service request in the model will have a ServiceRequestOb observer associated to it. Considering that a service request is the relation among a server and a workload that requests its service, the data monitored by this observer represents the performance relation among them. In this way, when we mention workloads (or jobs belonging to them) and servers in the explanation of the attributes, we mean the workloads (or jobs) and servers associated to the service request:

- served. It counts the number of jobs processed by the server.
- timeBusy. It stores the time the server has been busy processing jobs.
- utilization. Percentage of the time (between 0 and 1) that the server has been busy.
- waitingTAcc. Used to calculate the average waiting time. It stores the addition of waiting times of all the jobs.
- waitingTAv. It monitors the average waiting time in the queue for all the jobs processed by the server, where the waiting time of a job is the time that elapses between the job arrives to the queue of the server and it starts being processed.
- serviceTAcc. Used to calculate the average service time. It stores the addition of service times of all the jobs.
- serviceTAv. It calculates the average service time of all the jobs processed by the server.
- residenteTAv. It stores the residence time average of the jobs processed by the server. The residence time of a job is the time elapsed between the job arrives to the queue of the server and the job leaves the server.
- throughput. It computes the throughput, i.e., the jobs processed by the server per unit of time.
- utilizTrace, waitTrace, servTrace, residTrace and thrTrace. For the performance metrics described, these attributes keep the traces of their values throughout simulation. They are used to determine when the simulation has to stop.

5.3 QNMs behavioral model

In this section we show the rules needed to model any kind of QNM. But before doing so we need to make two small clarifications. In the first place, we have a rule that whenever the trans attribute of OWLnT and SRnT objects has the value -1, it assigns them a random value between 1 and 10000. It is used for the jobs transitions. Secondly, we need to explain how attributes wklds, tS and aS in TServiceRequests are used. These attributes are sequences of integers that are used to model the flow of jobs among servers. For each job present in the network, they store its identifier (in sequence wklds), the time it entered the network (in tS - timeStamp) and the time it arrived to the Server where it currently is (in aS - arrivalServer). In this way, at a certain point in the simulation and at a certain server, we could have, for example, the following sequences for these attributes: wklds: Sequence(2, 4, 1), tS: Sequence(13, 25, 4) and aS: Sequence(21, 25, 32). Attributes tS and aS are used for the calculation of some performance metrics. Please note that the order matters since the position of an element in any of the sequences contains information about a job to which the value in the same position in the other sequences gives information too.

Now let us explain the *e-Motions* rules that describe the behavior of QNMs. The first one is the EnterOpenWLFnT rule (shown Fig. 9), which models how jobs belonging to a workload of type OWLnT enter the network. It shows how the job can transit to more than one server once in enters the system by means of the Source node. We can see in the figure that in both the LHS and RHS patterns we have the OpenWorkload to which the job belongs, the Server to which the job transits, the ServiceRequest that relates both of them, and the Source node at which jobs belonging to the OpenWorkload enter. We can also see the relations between these objects, according to the xPMIF-eMotions metamodel (Fig. 7).

There are two variables in the EnterOpenWLFnT rule, duration and pos. The former specifies the duration of the rule. This rule if for OpenWorkloads whose arrival distribution is Poisson. Consequently, the duration of the rule is determined by an Exponential distribution where the parameter is the inverse of the parameter of the Poisson. The parameter of a

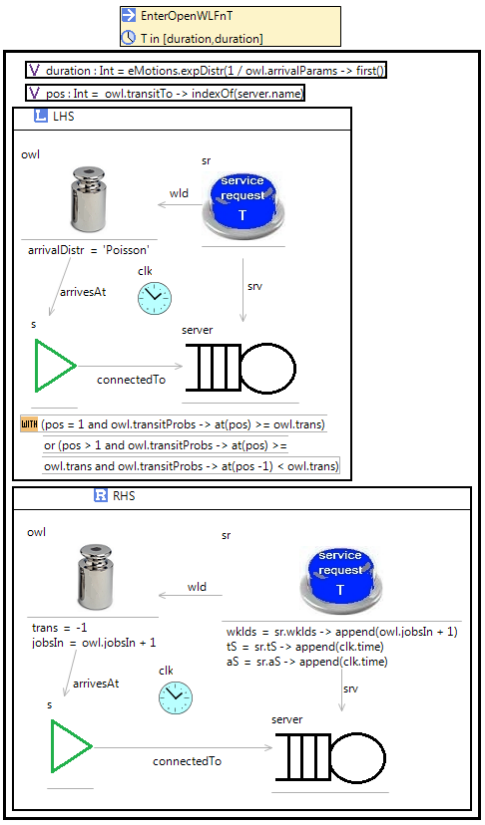


Fig. 9 EnterOpenWLFnT rule

Poisson determines the average jobs arrival per unit of time. An Exponential with the inverse of the parameter of the Poisson determines the average time elapse between the arrival of two jobs. If the OpenWorkload has a different probabilistic distribution, we only need to change the value of duration. The latter variable is used for determining the Server where the job transits. It contains the position that the server has in the sequence transitTo of the OWLnT. It is used for the OCL condition needed to trigger the rule. This condition basically states that the Server with which this rule is doing the matching corresponds to the Server to which the job has to transit as specified by the attribute trans of the OWLnT. In the RHS of the rule we see how the trans attribute of the OWLnT is -1 (so that a new random value will be assigned to it) and its jobsIn attribute has been incremented. The attributes of the TServiceRequest are updated so that a new identifer is given to the new job (wklds sequence) and the current time elapse is assigned to sequences tS and aS.

We have a very similar rule, named EnterOpenWLF1T, which contains the same as EnterOpenWLFnT excepting the variable pos and the condition in the LHS of the rule. The rule is used for OpenWorkloads of type OWL1T, i.e., OpenWorkloads that always transit to the same Server when they enter the Source node.

The TransitJobsnT rule (Fig. 10) models the transition of jobs among servers (and also from a Server to a Node — of type SinkNode). Jobs can belong to either OpenWorkloads or ClosedWorkloads, so this rule is used for both. Specifically, the TServiceRequest tagged as srS, i.e., the service request of the source server in the transition, is of type SRnT in this rule. It means that jobs have different nodes to transit to. There is a similar rule, named TransitJobs1T, which is for TServiceRequests of type SR1T. The rule is basically the same as TransitJobsnT but without the pos variable and the second condition in the LHS. They are both for determining the Node to which jobs have to transit to, as explained in the EnterOpenWLFnT rule. The duration of the rule is also specified as described before. In the rule shown in Fig. 10, the TServiceRequest follows an Exponential distribution. If it followed a Gamma distribution, for example, then the duration attribute would be as follows, because it has two parameters:

```
duration : Int =
eMotions.gammaDistr(srS.params->at(1),srS.params->at(2))
```

In any case, all this is automatically generated from the xPMIF model provided by the user.

In the LHS of the rule we see all the objects needed for this rule to be triggered: the source Server, the target Node (either a Server or a SinkNode), the Workload to which jobs belong, the TServiceRequests associated to the mentioned elements, and the Observers whose attributes will be updated in the RHS of the rule. In the RHS of the rule we see how the three sequences representing the jobs in the source TServiceRequest (srS) are updated with the elimination of the corresponding jobs, and these sequences in the target TServiceRequest (srT) and updated too with the addition of the same jobs. The helpers used are shown in Listing 1. The trans attribute of srS is set to -1 so a new random value will be given to it for the next triggering of the rule. The attributes of the observer of the source TServiceRequest are updated as it corresponds. Basically, the updating consists on calculating the new value for the attributes taking into account the previous value and the jobs that are being processed in the rule. In this way, waitingTAcc and serviceTAcc attributes update their values with the waiting time and the service time of the jobs processed, respectively. Attributes waitingTAv and serviceTAv are updated using those values. The value of the throughput is calculated with the new total number of jobs served and the current time elapse. The timeBusy attribute is updated with the duration of the rule plus its old value, and the utilization is updated with these and the current time elapse. The residenceTAv is also updated. All the helpers are shown in Listing 1. Regarding the observer associated with the source Server (sOb), their attributes to update the average queue length use the helper lengthQA, shown in Listing 1.

Listing 1 Helpers used in TransitJobsnT and TransitJobs1T rules

```
context Sequence::remJobs(n : Integer): Sequence(Integer)
  body: if n >= self->size() then Sequence{} else self
 ->subSequence (n+1, self->size()) endif;
context Sequence::appendTs(q : Integer, ts : Sequence,
  wklds2 : Sequence, wklds : Sequence): Sequence(Integer)
  body: self->select(i | self->indexOf(i) <= q)->
  iterate (t ; acc : Sequence = ts | acc->append(t))->
  subSequence(1, wklds2->size() + (wklds->  select(i
  | wklds->indexOf(i) <= q)->size()));
context Sequence::appendAs(q : Integer, arr2 : Sequence,
  time : Integer, wklds2 : Sequence, wklds : Sequence):
  Sequence(Integer)
```

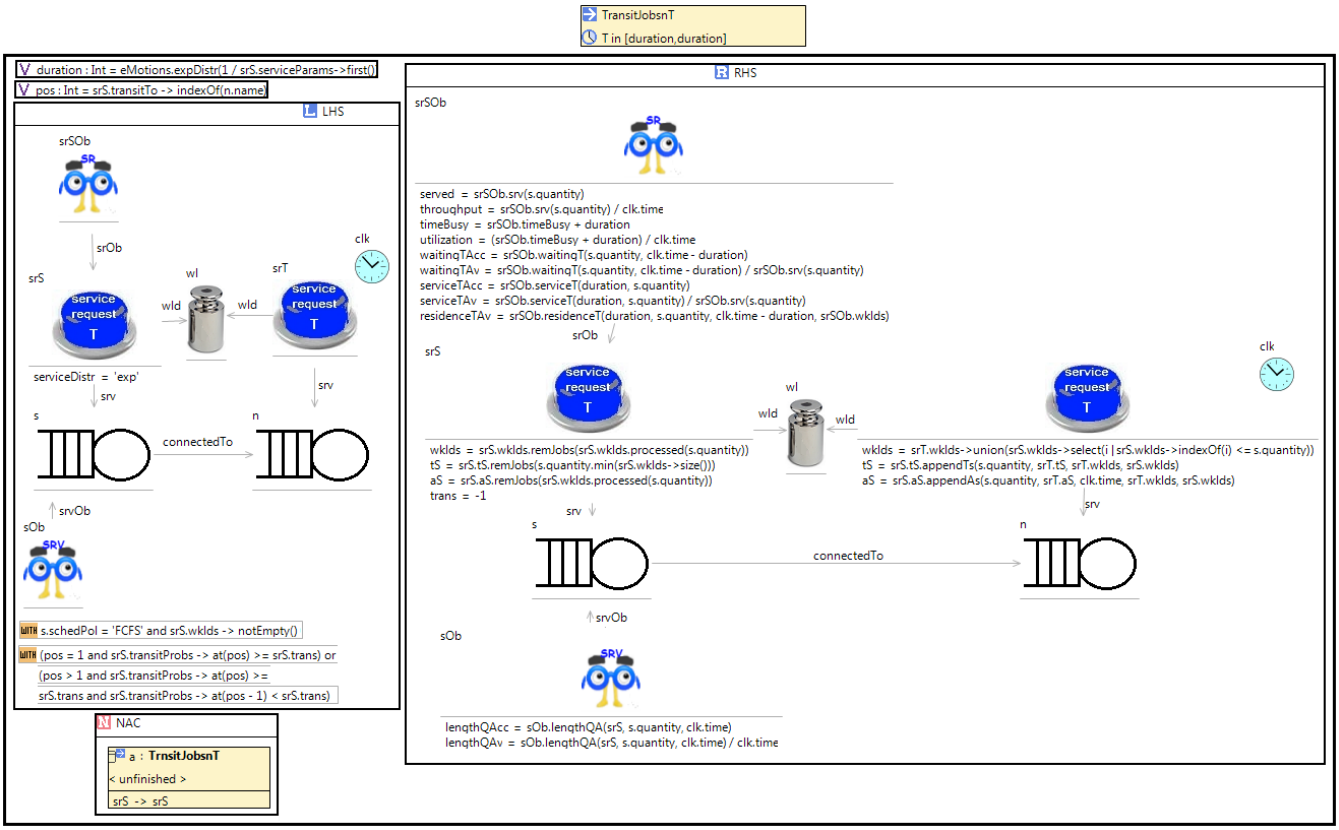


Fig. 10 TransitJobsnT rule

```
  body: self->select(i | self->indexOf(i) <= q)->
  iterate (t ; acc : Sequence = arr2 | acc->append(time))
 ->subSequence(1, wklds2->size() + (wklds->select(i |
  wklds->indexOf(i) <= q)->size()));
context ServerOb::lengthQA(sr : TServiceRequest, q :
  Integer, time : Integer): Integer
  body: self.lengthQAcc + (q . min(sr.wklds->size()))
  * time - sr.aS->select(i | sr.aS->indexOf(i) <= q)
 ->subSequence (1, q.min(sr.wklds->size()))->sum();
context ServiceRequestOb::srv(q : Integer): Integer
  body: self.served + self.srOb.wklds.processed(q);
context ServiceRequestOb::waitingT(q : Integer,
  t : Integer): Integer
  body: self.waitingTAcc + (q.min(self.srOb.wklds->
  size())) * t - self.srOb.aS->select(j | self.srOb.aS
 ->indexOf(j) <= q)->iterate(i; acc : Sequence =
  Sequence{} | acc->append(i.min(t)))->
  subSequence(1, q.min(self.srOb.wklds->size()))->sum();
context ServiceRequestOb::serviceT(d : Integer,
  q : Integer): Integer
  body:
    self.serviceTAcc+(d*self.srOb.wklds.processed(q));
context ServiceRequestOb::residenceT(d : Integer, q :
  Integer, time : Integer, wklds : Sequence): Real
  body: (self.waitingT(q, time) / self.srv(q)) +
  (self.serviceT(d, q) / self.srv(q));
```

Although the code may look a bit verbose, the calculation of these performance parameters is quite straightforward. In this way, the mean waiting time is calculated as the sum of waiting times divided by the number of jobs that received service. In the same way, the mean service time is the total service time divided by the number of jobs that completed service. Same thing to calculate the mean residence time: it is the total residence time divided by the number of jobs that completed service. The calculation of the mean queue length in the Server is not done in the same way. Thus, if q_j is the queue length at the jth event (from a total of n events) that resulted in a queue length change, the average of q_j's does not give the mean queue length. That is, the mean queue length should not be calculated as $\sum_{j=1}^{n} \frac{q_j}{n}$. Instead, a time average of the queue length process should be used, that is, the mean queue length should be $\frac{1}{T}\int_0^T QueueLength(t)\, dt$. This is the way in which we have calculated it. An example of this can be seen in Fig. 11. Imagine that three jobs arrive at time $t = 0$. Two of them depart at $t = 1$, while the third one departs at $t = 4$. There are three events: one arrival and two departures. The queue lengths at each of the events are 3, 1, and 0, respectively. As explained, the average queue length is not $(3 + 1 + 0)/3$, that is, $\frac{4}{3}$. Instead, the area under the queue length curve is 6. Consequently, the correct average queue length is $\frac{6}{4}$, i.e., $\frac{3}{2}$.

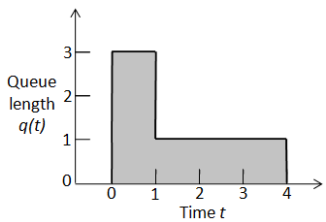


Fig. 11 Example of time average of queue length

Continuing with the explanation of the TransitJobsnT and TransitJobs1T rules, their NACs forbid the application of the rules if the source TServiceRequest (srS) is already participating in an action of this type. The scheduling policy of the source Server is FCFS (First Come First Served). Basically, the number of jobs that are processed by the rule is the minimum value between the quantity attribute of the source Server and the number of jobs in the queue. If the scheduling policy is IS (Infinite Server), the rule is very similar, with the only need to change the quantity of jobs that are processed. Thus, all the jobs in the queue are processed.

The TransitJobsnT rule shown in Fig. 10 is used when there is only one kind of Workload in our model. If there are more than one Workload, we need to add the NAC shown in Fig. 12, so that the job that arrived first to the source Server is the one processed. In fact, the NAC forbids the rule triggering if there is a different TServiceRequest from the one in the rule matching and that is pointing to the same source Server and Workload and whose first job in the queue arrived earlier than the first job in the TServiceRequest of the matching.

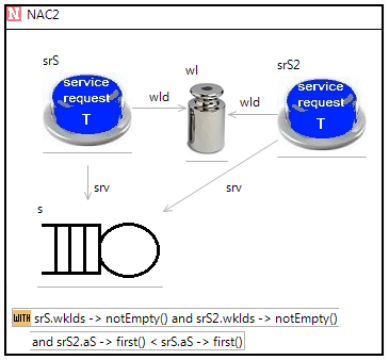


Fig. 12 NAC in case there are more than one Workloads

The instantaneous ExitOpenWLF rule (Fig. 13) models the exit of jobs from the network. Consequently, it is applied only over OpenWorkloads. The rule is instantaneous. The reason why there is a reference from TServiceRequests to Nodes (instead of only to Servers) in the metamodel used in *e-Motions* (Fig. 7) is to associate a TServiceRequest to the SinkNode. In this way, when this TServiceRequest contains jobs, this rule is fired and the corresponding attributes in the OpenWorkload are updated. The jobsOut attribute of the OpenWorkload is updated with the helper jobsO (Listing 2). The throughput of the observer is updated using this value. The mean response time is calculated in the respTimeAv attribute and uses the value of respTimeAcc. This, in turn, calls the helper ResponseTWL. Regarding the mean number of jobs, it is calculated following the same approach as the mean queue length explained before. Thus, we take into account the jobs exiting the network and how long they have stayed within this. Its calculation is done with the jobsA helper. The jobs in the TServiceRequest are deleted, since they have exited the network.

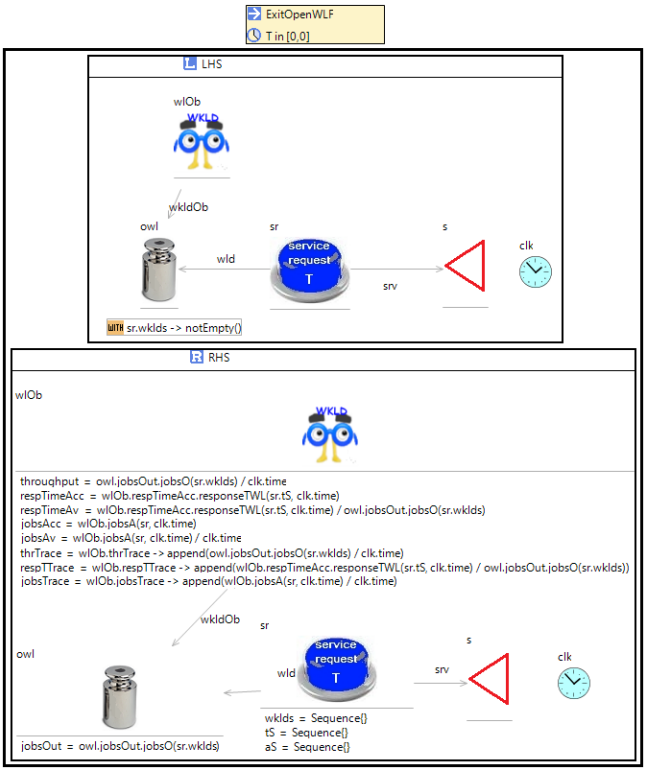


Fig. 13 ExitOpenWLF Rule

Listing 2 Helpers used in ExitOpenWLF rule

```
context Integer::jobsO(wklds : Sequence): Integer
  body: self + wklds->size()
context Integer::responseTWL(ts : Sequence,
  t : Integer): Integer
  body: self + (t * ts->size()) - ts->sum()
context WorkloadOb::jobsA(sr : TServiceRequest,
  time : Integer): Integer
  body: self.jobsAcc + sr.wklds->size()*time
        - sr.tS->sum()
```

Regarding the thrTrace, RespTTrace and jobsTrace attributes, the new value calculated for throughput, mean response time and jobs average is appended to each of the sequences, respectively. In this way, a new value is added to these sequences every time a job (or more than one jobs) leave the system.

There is a similar rule to the ExistOpenWLF one to update the attributes of observers associated to ClosedWorkloads. The attributes are the same excepting the one for the mean number of jobs, which is no longer necessary. In that rule, instead of having the SinkNode of the OpenWorkload, we have the centralSrv associated to the ClosedWorkload. It is equivalent to the thinkDevice in Fig. 5.

Finally, we have another rule, UpdateTraces, to update the attributes for traces in the other observers. They are updated either when jobs leave the system (in OpenWorkloads) or when jobs arrive to the centralSrv (for ClosedWorkloads). The rule is instantaneous.

It is important to remind that users do not need to create these rules, they are already created in *e-Motions* and apply to all QNMs. In fact, they can be seen as providing a behavioral

semantics of QNMs by explicitly specifying the behavior of QNM in a model with well-defined semantics [39].

6 xQNM: A Tool for Drawing and Simulating QNMs

Our proposal is supported by a tool, called xQNM (standing for *executable*-QNM), which provides users with a graphical interface where queuing network models conforming to PMIF or xPMIF metamodels can be drawn. Furthermore, PMIF models (XML files according to the PMIF 2 metamodel in Fig. 2) can be imported (either because they already existed or because they have been produced by other tools). These models are then translated to the QNMs behavioral representation described in Section 5 to obtain performance indicators. Results are returned as either E-core models or *cvs* format that can be used by spreadsheet applications to display and analyze them, perform further calculations, etc. This section explains the possibilities offered by the tool.

6.1 QNMs graphical definition in xQNM

In Section 4.1 we presented GMF, which has been used to develop the graphical editor of our tool. In Fig. 14 we can see a snapshot of our editor. Objects (OpenWorkloads, ClosedWorkloads, Servers, WorkUnitServers, SourceNodes, SinkNodes, WorkUnitServiceRequests, TimeServiceRequests and DemandServiceRequests) can be selected from the menu on the right and be placed on the main panel. When the user is placing objects in the main panel, she has to keep in mind that she is defining a new model conforming to the metamodel shown in Fig. 5, which, according to Section 4.1, is the abstract syntax of a DSL for defining queuing network models.

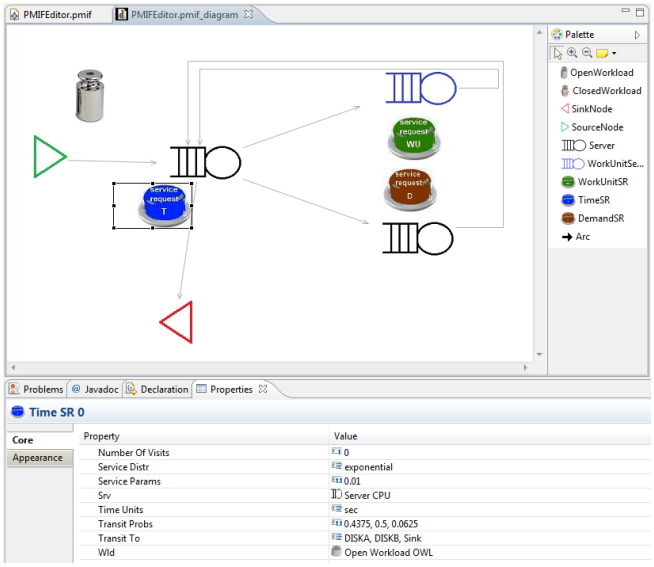


Fig. 14 xQNM Graphical Editor

Objects properties (attributes and references) are specified in the lower panel. The user familiarized with the PMIF 2 metamodel (Fig. 2) has to be aware of the differences with our metamodel (Fig. 5), explained in Section 3. To assign values to the sequences of transitions, the user has to select the object and click on the attribute in the lower panel. A new window where the values can be introduced is shown in Fig. 15. Probabilistic distributions are specified in some attributes of type ProbDistribution enumerate. When we have an object selected and click one of these attributes in the lower panel, we get a drop down list with the probability distributions belonging to the enumerate type (Fig. 16(a)). As for the parameters of the distributions, they are specified in an attribute of type sequence (arrivalParams, serviceParams and thinkTimeParams) and they have to be introduced following the order mentioned in Sec. 3. Regarding references, they appear in the same way as attributes in the lower panel. When a reference is clicked, a drop down list appears with the objects belonging to the target class of the reference (Fig. 16(b)). So, in order to include a reference that links the object with another one, the target object has to be created.

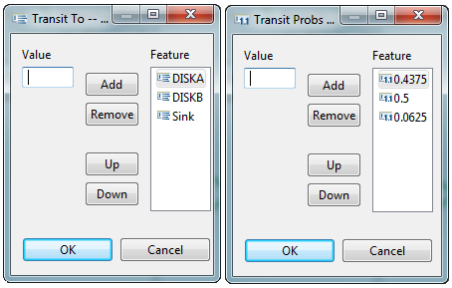
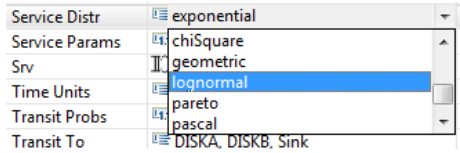
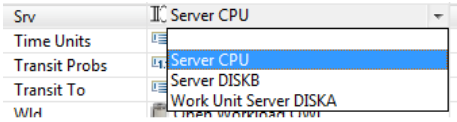


Fig. 15 Assigning values to sequences



(a) Specifying the distribution.



(b) Selecting the Server associated to the ServiceRequest.

Fig. 16 Drop down lists in the lower panel.

When a new model is created in our tool (in fact, in any one developed with GMF), there are two files. One of them is the one edited with the graphical tool. The other one is a tree

view representation of the model. The user can select any of them in a panel on the left.

6.2 QNMs exportation in xQNM

This subsection refers to the ovals 2 and 3 in Fig. 6 and the ATL transformation from 2 to 3.

Once a queuing network model is defined with the graphical editor, it can be exported to an XML file where the model is represented as explained in [14] but with an extension for allowing different probabilistic distributions. In this way, there are not ArrivalRate, ServiceTime and ThinkTime attributes anymore; but ArrivalDistr, ServiceDistr and ThinkDistr. For the objects that contain any of these attributes, they also have to contain one or more attributes named Param that specify the parameters of the distributions, in the order specified in Section 3. For instance, let us consider the example shown in Fig. 3 and described in Section 3 of an open queuing network model with a CPU and two disks A and B. Distributions for service times are supposed to be Exponential, and Poisson for arrival times. In Listing 3 we can see an XML file that has been exported from the definition of this open queuing network model in our tool. The XML is similar to the XML files conforming to the PMIF 2 metamodel with the corresponding extensions for transitions and probability distributions.

Listing 3 xPMIF XML File

```
<QueueingNetworkModel Name="Jain572" Description=
 "Extended XML PMIF" Date-Time="040711">
  <Workload>
    <OpenWorkload WorkloadName="OWL" ArrivesAt="Source"
     DepartsAt="Sink" ArrivalDistr="Poisson"
     TimeUnits="sec">
       <Transit Probability="1.0" To="CPU"/>
       <Param Value="3.0"/>
    </OpenWorkload>
  </Workload>
  <Node>
    <Server Name="CPU" Quantity="1"
     SchedulingPolicy="FCFS"/>
    <Server Name="DISKB" Quantity="1"
     SchedulingPolicy="FCFS"/>
    <WorkUnitServer Name="DISKA" Quantity="1"
     SchedulingPolicy="FCFS" TimeUnits="sec"
     ServiceDistr="Gamma">
       <Param Value="0.5"/>
       <Param Value="2.0"/>
    </WorkUnitServer>
    <SourceNode Name="Source"/>
    <SinkNode Name="Sink"/>
  </Node>
  <ServiceRequest>
    <DemandServiceRequest ServiceDemand="2592.0"
     TimeUnits="sec" WorkloadName="OWL"
     ServerID="DISKB" NumberOfVisits="86400">
       <Transit Probability="1.0" To="CPU"/>
    </DemandServiceRequest>
    <WorkUnitServiceRequest WorkloadName="OWL"
     ServerID="DISKA">
       <Transit Probability="1.0" To="CPU"/>
    </WorkUnitServiceRequest>
    <TimeServiceRequest TimeUnits="sec"
     WorkloadName="OWL" ServerID="CPU"
     ServiceDistr="Exponential">
       <Param Value="0.01"/>
       <Transit Probability="0.4375" To="DISKA"/>
       <Transit Probability="0.5" To="DISKB"/>
       <Transit Probability="0.0625" To="Sink"/>
    </TimeServiceRequest>
  </ServiceRequest>
  <Arc FromNode="Source" ToNode="CPU"/>
  <Arc FromNode="CPU" ToNode="DISKA"/>
  <Arc FromNode="CPU" ToNode="DISKB"/>
  <Arc FromNode="CPU" ToNode="Sink"/>
  <Arc FromNode="DISKA" ToNode="CPU"/>
  <Arc FromNode="DISKB" ToNode="CPU"/>
</QueueingNetworkModel>
```

6.3 QNMs importation in xQNM

This subsection describes ovals 1, 2 and 3 in Fig. 6 and the ATL transformations from 1 and 3 to 2.

The xQNM tool also offers the possibility to import PMIF models defined in XML files. Both PMIF 2 models (Fig. 2) and xPMIF models in XML format (see Section 6.2) can be imported. When importing PMIF 2 models, ServiceTime and ThinkTime attributes are translated into serviceDistr and thinkDistr attributes, respectively, of type Exponential. Attributes named serviceParams and thinkParams are also created as sequences of one element whose values are those in the ServiceTime and ThinkTime attributes, respectively. The same happens with the ArrivalRate attribute in PMIF 2, which is translated to an arrivalDistr attribute, this time of type Poisson.

It is also possible to import XML files with models of type xPMIF to the xQNM tool. The ATL transformation used is the opposite as the one used for the exportation of xPMIF models.

When XML files containing either PMIF or xPMIF models are imported into our tool, a file containing the new model generated is created. This file only accepts the visualization in the tree view form. From this file, the user can generate another file where the model is deployed in the graphical editor. The elements will appear in a random position in the graphical editor (since PMIF 2 XML files do not store objects position, and neither do xPMIF XML files), so the user only has to arrange the objects as preferred.

6.4 From QNMs in xQNM to QNM generic behavior in e-Motions

Once the user inserts a model within the xQNM tool either by drawing it with the graphical interface or by importing it, it is translated to its structural and behavioral models. This is done by the ATL transformation from oval 2 to ovals 4 and 5 in Fig. 6.

The transformation has two main parts, the generation of the structural model and the generation of the behavioral model. For the former, the transformation takes the xPMIF model conforming to the xPMIF metamodel and transforms it into another model conforming to the metamodel shown in Fig. 7. This transformation takes into account the differences between the two metamodels as described in Section 5.1. Regarding the latter, the transformation identifies the type of queuing network the model is (open or closed) and selects the appropriate rules among the ones presented in Section 5.3. Necessarily, all those rules are input parameters in the ATL transformation (plus the xPMIF model). At the same time, the transformation also modifies some features in the rules if it is

necessary. For example, for a Gamma distribution in a TServiceRequest object, the variable duration in rule TransitJobsnT (Fig. 10) has to be accordingly written. Consequently, as previously mentioned, all the rules are input parameters of the ATL transformation (so they are static), but some of them are also slightly modified in the ATL transformation (so they are dynamically edited by the transformation). The xQNM tool is available from [40].

7 Simulation and Discussion

Once we have the behavioral dynamics of a QNM specified in *e-Motions*, we are able to simulate it. In fact, our environment supports the translation of the specifications (ATL transformations from ovals 4 and 5 to oval 6 in Fig. 6) into the corresponding formal specifications in Real-Time Maude [41, 42]. The objective of such formal specifications is to provide formal semantics to the visual specifications of the system. In [35] we showed some examples of analysis that can be performed on the Maude specifications. Furthermore, Maude rewriting logic specifications are executable, and therefore they can be used as a prototype of the system and to run simulations.

In Maude, the result of a simulation is the final configuration of objects reached after completing the rewriting steps, which is nothing but a model. The semantic mapping as well as the transformation process back and forth between the *e-Motions* and Real-Time Maude specifications is described in detail in [35], although it is completely transparent to the *e-Motions* (and so xQNM) user. The user, consequently, is completely unaware of the Maude rewriting engine performing the simulation.

A very important advantage of our approach is that observers are also objects of the system, and therefore we can retrieve the values of their attributes after the simulation is conducted to know how the system behaved. In fact, this is crucial for the approach we are presenting here.

If we could see with an animation tool a queueing network model being running, what we would see are the jobs flowing from resource to resource, entering the network, leaving the network, and waiting in queues. In our specifications, if we removed the observers, this is what we would have when we simulate the system. The aim of the observers is to monitor performance metrics in the network as jobs flow within it. In this way, the state of observers changes as the simulation moves forward.

An important issue concerning queuing network simulations is the stopping criteria. The simulation should stop at a certain point where the performance parameters are reliable enough. To be able to reach that point at some time in the simulation, the Little's Law has to be satisfied in the network. It relates the mean number of jobs in the system with the mean time spent within it as follows: Mean number in the system = arrival rate x mean response time. This relationship applies as long as the number of jobs entering the system is equal to those leaving it. It implies that the jobs arrival to the network is slower than the job service in the resources. Otherwise, the network would overflow as the number of jobs within it would increase as long as the simulation moves forward. Furthermore, in a QNM simulation, this law has to be accomplished in order to reach the steady state at any point. This law is also necessary for QNMs analysis methods.

It is important to properly choose the length of the simulation. Thus, if the simulation is too short, the results will be probably unreliable. On the other hand, if the simulation is too long, computing resources and users' time will be unnecessarily wasted. In most simulations, only the performance after the system reaches a stable state is of interest. In this way, the initial part, also called transient state or warm-up period, should not be included in the final computations. The problem of identifying the end of the transient state receives the name of transient removal. The main difficulty with the transient removal is that there is not an homogeneous way to determine when the transient state finishes and the steady state starts. Thus, some methods for transient removal, which are nothing but heuristics, exist: long runs, proper initialization, truncation, initial data deletion, moving average of independent replications, batch means.

In our approach, we discard the method of long runs because a simulation should not be run any longer than is absolutely necessary. Regarding the heuristic of proper initialization, it requires starting the simulation in a state close to the expected steady state. We cannot follow this approach since, in fact, we do not know when the steady state is reached. Truncation method is based on the assumption that the variability during the steady state is less than during the transient state, which is normally true. It considers that the data in the transient state is continuously either increasing or decreasing. This method is not good for our approach because there are some peeks in the transient states of our simulations. The methods of initial data deletion and moving average of independent replications require studying the overall average after some of the initial observations are deleted from the sample. These methods apply over several replications, which differ only in the seed values used in the random number generators, of a fixed size. The problem, again, is how to determine the length of the replications.

To determine when a simulation has to stop, we have applied the method of the batch means with some modifications. The method of batch means requires running a very long simulation and later dividing it up into several parts of equal duration, which are called batches. The mean of the observations within each batch is called the batch mean. The method requires studying the variance of these batch means as a function of the batch size. We do not run a very long simulation to later diving it up. We, instead, apply the method sometimes during the simulation as it moves forward.

In our *e-Motions* approach, we store the values of the performance parameters in traces to be able to apply this method over them. We apply the batch means method over the traces every time that 100 new jobs leave the system (or, in the case of closed networks, when they complete a cycle). Generally, we consider that a simulation has reached the steady state

when the variance of the batch means is in the order of 10^{-6} (of course, this parameter can be changed). In Fig. 17 we can see a chart with the variance (Y-axis) plotted as a function of batch sizes (X-axis).

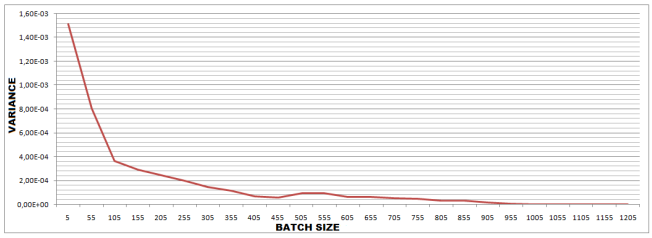


Fig. 17 Method of the Batch Means

In this example, batch sizes have been increased in steps of 50. In the data plotted, the variance goes below 10^{-6} in the step from 905 to 955. Thus, we can conclude that the steady state in these data starts when 955 jobs have exited the system (or done a cycle in a closed network).

We run independent simulations and, for every simulation, we apply the batch means method to their traces. When the variances of the simulations reach the desired values, the simulations stop (in some simulations, the steady state may be reached earlier than in others). The current value of the attributes of the observers when a simulation stops, which in fact is the same as the last values in the traces, is the one that we pick in every simulation. These values, as we are in the steady state, are very close to the real value. The values in some simulations will be slightly above the real values while the values in other simulations will be slightly below them. For this reason, we do the average of these values, and the result is returned to the user.

When all the simulations have stopped and the performance results are to be returned to the user, an ATL transformation is applied to the data. This transformation is the one from oval 7 to 8 in Fig. 6. Its aim is to return the data in a format that can be easily consulted, managed and manipulated by the user. This is why we have chosen a *csv* format, which is readable by spreadsheet applications. In Figure 18 we can see the results returned by our approach for our open queue network example.

8 Conclusions and Future Work

The main contribution of our paper is the generic approach proposed to model QNMs using behavioral rules. This can also be seen as a behavioral semantics for QNMs since the way in which QNMs work can be precisely described by the *e-Motions* behavioral rules.

The behavioral specifications can be simulated by following an MDE approach that uses standard model transformation techniques. Thanks to the use of observers in our specifications, performance metrics for the QNMs can be monitored in the simulations. Our approach accepts many probabilistic distributions for arrival and service times (Exponential,

SERVICE REQUESTS	
ServiceRequest	owl, CPU
Utilization	0,47365
Throughput	47,46225
Waiting Time	0,00823
Service Time	0,00992
Residence Time	0,01815
Served	38755

ServiceRequest	owl, DISKA
Utilization	0,41726
Throughput	20,76636
Waiting Time	0,01347
Service Time	0,02012
Residence Time	0,03359
Served	16850

ServiceRequest	owl, DISKB
Utilization	0,71737
Throughput	23,83625
Waiting Time	0,07586
Service Time	0,03017
Residence Time	0,10603
Served	19405

SERVERS	
Server	CPU
Length queue	1,62958

Server	DISKA
Length queue	1,48269

Server	DISKB
Length queue	3,86924

WORKLOADS	
OpenWorkLoad	owl
Response Time	1,37598
Throughput	3,06168
Jobs Average	5,96572

Fig. 18 Results of Jain572

Uniform, Weibull, Geometric, Normal, Gamma, Chi-square, etc.). Heuristic techniques have been used to determine when simulations have to stop. That is, in order to obtain reliable performance indicators, simulations stop after the warm-up period finishes and the steady state is reached.

We have also presented the xQNM tool that allows drawing QNMs and then translate them to their behavioral representation and simulate them. Existing PMIF models can also be imported for simulation with xQNM.

In our approach, the user can select a number of simulations and we now return the average of the results of these simulations. We are now working on returning not just the mean value, but the probabilistic distributions that these results follow.

Another interesting task that we leave for future work is the distribution of the simulations among several machines (or processors). Now, the simulations are run locally in the user machine. We could significantly improve the performance by using a concurrent and distributed solution that uses a *process farm* approach.

References

1. Norris, J.R.: Markov Chains. University Press (1997)
2. Kartson, D., Balbo, G., Donatelli, S., Franceschinis, G., Conte, G.: Modelling with Generalized Stochastic Petri Nets. 1st edn. John Wiley & Sons, Inc., New York, NY, USA (1994)
3. Bause, F., Kritzinger, P.S.: Stochastic Petri nets - An introduction to the theory (2. ed.). Vieweg (2002)
4. Brinksma, E., Hermanns, H.: Process algebra and Markov chains. In: Lectures on formal methods and performance analysis. Springer-Verlag New York, Inc., New York, NY, USA (2002) 183–231
5. Clark, A., Gilmore, S., Hillston, J., Tribastone, M.: Stochastic process algebras. In: Proceedings of the 7th International Conference on Formal Methods for Performance Evaluation. SFM'07, Berlin, Heidelberg, Springer-Verlag (2007) 132–179
6. Denning, P.J., Buzen, J.P.: The Operational Analysis of Queueing Network Models. ACM Comput. Surv. **10** (1978) 225–261

7. Hirel, C., Sahner, R.A., Zang, X., Trivedi, K.S.: Reliability and Performability Modeling Using SHARPE 2000. In: Proceedings of the 11th International Conference on Computer Performance Evaluation: Modelling Techniques and Tools. TOOLS '00, London, UK, Springer-Verlag (2000) 345–349
8. Bolch, G., Kirschnick, M.: The Performance Evaluation and Prediction SYstem for Queueing NetworkS - PEPSY-QNS. Technical Report TR-I4-94-18, University of Erlangen-Nuremberg, Germany (1994)
9. Gunther, N.J.: Analyzing Computer System Performance with Perl::PDQ. Springer Verlag, Berlin (2005)
10. Chereddi, C.: Mean Value Analysis for Closed, Separable, Multi Class Queuing Networks with Single Server & Delay Queues. Technical report, University of Illinois, Urbana-Campaign, (USA)
11. Brenner, L., Fernandes, P., Sales, A.: MQNA - Markovian Queueing Networks Analyser. In: MASCOTS, Porto Alegre, Brazil (2003) 194–199
12. Veran, M., Potier, D.: QNAP2: A Portable Environment for Queueing System Modelling. In: Proceedings of the International Conference on Modelling Techniques and Tools for Performance Analysis. (1984) 5–24
13. Wüchner, P., de Meer, H., Barner, J., Bolch, G.: A brief introduction to MOSEL-2. In: Proceedings 13th GI/ITG Conference on Measuring, Modelling and Evaluation of Computer and Communication Systems, MMB, Nürnberg, Germany, VDE Verlag (2006) 469–472
14. Smith, Connie U. and Lladó, Catalina M. and Puigjaner, Ramon: Performance Model Interchange Format (PMIF 2): A comprehensive approach to Queueing Network Model interoperability. Performance Evaluation **67**(7) (2010) 548 – 568
15. Smith, Connie U. and Lladó, Catalina M.: Performance Model Interchange Format (PMIF 2.0): XML Definition and Implementation. In: Proceedings of the The Quantitative Evaluation of Systems, First International Conference, Washington, DC, USA, IEEE Computer Society (2004) 38–47
16. García, D., Lladó, C.M., Smith, C.U., Puigjaner, R.: Performance Model Interchange Format: Semantic Validation. In: Proceedings of the International Conference on Software Engineering Advances, Washington, DC, USA, IEEE Computer Society (2006) 47–52
17. Smith, C.U., Williams, L.G.: A performance model interchange format. Journal of Systems and Software **49**(1) (1999) 63–80
18. Lavenberg, S.S.: A perspective on queueing models of computer performance. Performance Evaluation **10** (1989) 53–76
19. Chang, K.C., Gordon, R.F., Loewner, P.G., MacNair, E.A.: The Research Queuing Package Modeling Environment (RESQME). In: Proceedings of the 25th conference on Winter simulation. WSC '93, New York, NY, USA, ACM (1993) 294–302
20. de Souza e Silva, E., Leo, R.: The TANGRAM-II Environment. In Haverkort, B., Bohnenkamp, H., Smith, C., eds.: Computer Performance Evaluation.Modelling Techniques and Tools. Volume 1786 of Lecture Notes in Computer Science. Springer Berlin / Heidelberg (2000) 366–369
21. de Souza e Silva, E., Figueiredo, D.R., Leão, R.M.: The TANGRAMII integrated modeling environment for computer systems and networks. SIGMETRICS Performance Evaluation Review **36** (2009) 64–69
22. Field, T.: JINQS: An Extensible Library for Simulating Multiclass Queuing Networks V1.0 User Guide. (2010) `www.doc.ic.ac.uk/~ajf/Research/manual.pdf`.
23. Berson, S., de Souza e Silva, E., Muntz, R.: An object oriented methodology for the specification of Markov models. In: Numerical Solution of Markov Chains. (1991) 11–36
24. Cheng, W.C.W.: The TANGRAM graphical interface facility (TGIF) manual. (2001) `http://bourbon.cs.ucla.edu:8801/tgif/`.
25. Smith, C.U.: Performance engineering of software systems. Addison-Wesley (1990)
26. Flatscher, Rony G.: Metamodeling in EIA/CDIF—meta-metamodel and metamodels. ACM Trans. Model. Comput. Simul. **12** (2002) 322–342
27. Smith, Connie U. and Lladó, Catalina M.: Performance Model Interchange Format (PMIF 2.0): XML Definition and Implementation. Technical report, L&S Computer Technology, Inc. Santa Fe (2007)
28. W3C: World Wide Web Consortium (2001) `http://www.w3c.org/`.
29. Jain, R.: The Art of Computer Systems Performance Analysis: techniques for experimental design, measurement, simulation, and modeling. Wiley (1991)
30. Stahl, T., Völter, M.: Model-Driven Software Development: Technology, Engineering, Management. Wiley, Chichester, UK (2006)
31. Eclipse: Graphical Modeling Framework. (2008) `http://www.eclipse.org/modeling/gmf`.
32. Czarnecki, K., Helsen, S.: Classification of Model Transformation Approaches. In: OOPSLA'03 Workshop on Generative Techniques in the Context of MDA. (2003)
33. Rivera, J.E., Durán, F., Vallecillo, A.: A Graphical Approach for Modeling Time-Dependent Behavior of DSLs. In: Proc. of VL/HCC'09, Corvallis, Oregon (US), IEEE Computer Society (2009)
34. Atenea: The e-Motions tool (2009) `http://atenea.lcc.uma.es/E-motions`.
35. Rivera, J.E., Vallecillo, A., Durán, F.: Formal Specification and Analysis of Domain Specific Languages using Maude. Simulation: Transactions of the Society for Modeling and Simulation International **85**(11/12) (2009) 778–792
36. de Lara, J., Vangheluwe, H.: Translating Model Simulators to Analysis Models. In: Proc. of FASE'08. Number 4961 in LNCS, Springer (2008) 77–92
37. OMG: Object Constraint Language (OCL) Specification. Version 2.2, Object Management Group. (2010) Document formal/2010-02-01.
38. Troya, J., Rivera, J.E., Vallecillo, A.: Simulating Domain Specific Visual Models by Observation. In: Proceedings of the 2010 Spring Simulation Multiconference. SpringSim '10, New York, NY, USA, ACM (2010) 128:1–8
39. Rivera, J.E., Durán, F., Vallecillo, A.: On the behavioral semantics of real-time domain specific visual languages. In: Proc. WRLA'10. Volume 6381 of LNCS., Springer (2010) 174–190
40. Troya, J., Vallecillo, A.: A Model-Driven Approach to Simulate Queuing Network Models (Resources). Universidad de Málaga. (2011) `http://atenea.lcc.uma.es/index.php/Main_Page/Resources/xQNM`.
41. Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Talcott, C.: All About Maude – A High-Performance Logical Framework. Volume 4350 of LNCS. Springer, Heidelberg, Germany (2007)
42. Ölveczky, P.C., Meseguer, J.: Semantics and pragmatics of Real-Time Maude. Higher-Order and Symbolic Computation **20**(1-2) (2007) 161–196